

HASHSIGS PRIMITIVE

FORMAL VERIFICATION SPECIFICATION

Quip Network

Quip Network,
Research Team

March 11, 2026

PART I: PROBLEM DEFINITION

1	hashsigns primitive	5
1.1	Intro	5
1.2	Hash-Based Signatures History	6
1.3	Hash-Based Signatures Features	7
1.4	Size and Time Comparison (I)	8
1.5	Size and Time Comparison (II)	9
2	Formal Verification	10
2.1	Introduction	10
2.2	Tamarin Prover	11
2.3	Game-based Proofs	12
2.4	Implementation Verification	13
2.5	UC Proofs	14

PART II: SYMBOLIC VERIFICATION

- 1 Tamarin Proof 16**
 - 1.1 Progress 16
 - 1.2 Code Demo 17
 - 1.3 Visualization Demo 18
 - 1.4 Next steps 19

PART III: GAME-BASED VERIFICATION

1	WOTS+ EasyCrypt proof	21
1.1	Main points (I) ¹	21
1.2	Games, Hybrids, Reductions ²	22
2	Toward HashSigs proof	23
2.1	Initial Steps	23
2.2	PrivateKey	24
2.3	Required PrivateKey Modifications	25
2.4	PublicKey	26
2.5	Required PublicKey modifications	27
2.6	Randomization Elements & Masks	28

¹Barbosa et al., 2024

²Barbosa et al., 2024

Part I

PROBLEM DEFINITION

HASHSIGS PRIMITIVE

INTRO

- ▶ Post-Quantum Hash-Based One-Time Signature Scheme
- ▶ Based on WOTS+ 2017 paper and RFC 8193
- ▶ Implemented in Rust, TS, Solidity.
- ▶ Currently our best choice
 - Implemented in Rust, TS, Solidity.
 - Relies only on Hash Function Assumptions.
 - Relies only on well-tested libraries.
 - Easy to develop and debug

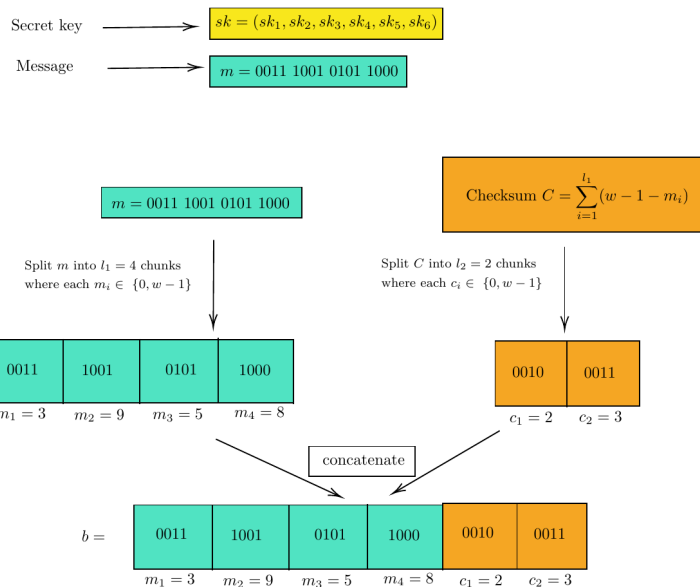


Figure. WOTS+ signature construction. (Srivastava et al., 2023)

HASHSIGS PRIMITIVE

HASH-BASED SIGNATURES HISTORY

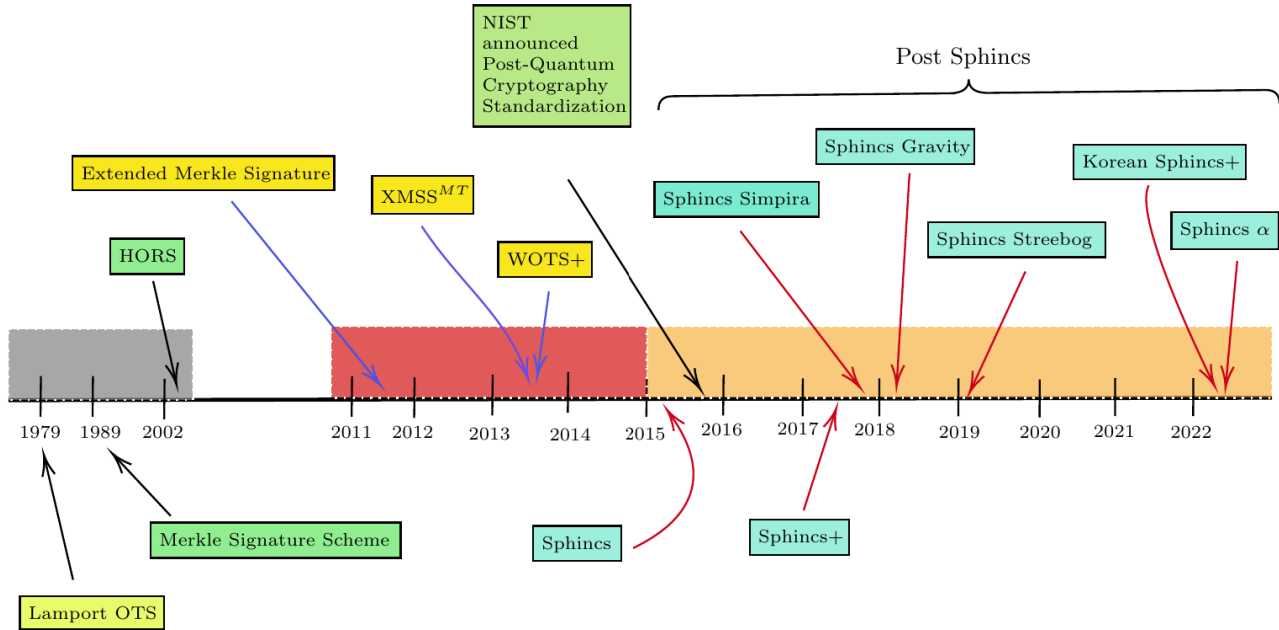


Figure. Hash-based signatures over time. (Srivastava et al., 2023)

HASH-BASED SIGNATURES FEATURES

- ▶ One-Time Signatures
- ▶ Few-time Signatures
- ▶ Stateless Signatures
- ▶ Stateful Signatures

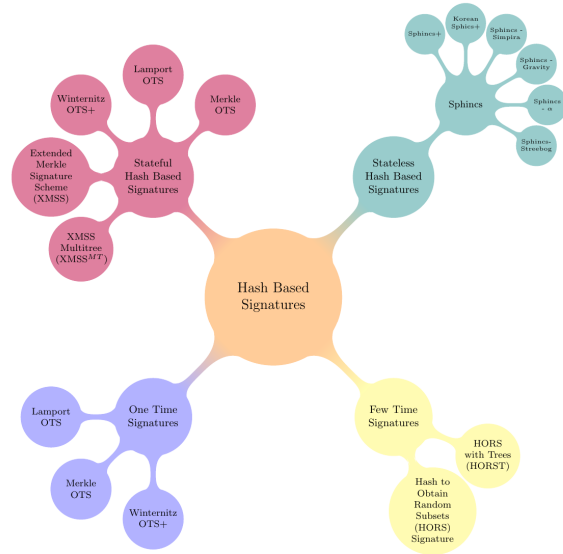


Figure. Hash-based signatures features. (Srivastava et al., 2023)

HASHSIGS PRIMITIVE

SIZE AND TIME COMPARISON (I)

Table. Signature Schemes Comparison

Scheme	Key Size	Signature Size	KeyGen/Sign Time	Verification Time
ECDSA (Baseline)	64 B - 96 B	64 B	Fast / Fast	Very Fast
HashSigs (ours)	32 B - 64 B	2,144 B	Very Fast / Moderate	Moderate
WOTS+	2,144 B - 2,144 B	2,144 B	Very Fast / Moderate	Moderate
Dilithium-2 ³	2,528 B - 1,312 B	2,420 B	Moderate / Moderate	Fast
Falcon-512 ⁴	1,281 B - 897 B	666 B (Avg.)	Moderate / Fast	Very Fast

³Gan and Yokubov, 2023

⁴Gan and Yokubov, 2023

HASHSIGS PRIMITIVE

SIZE AND TIME COMPARISON (II)

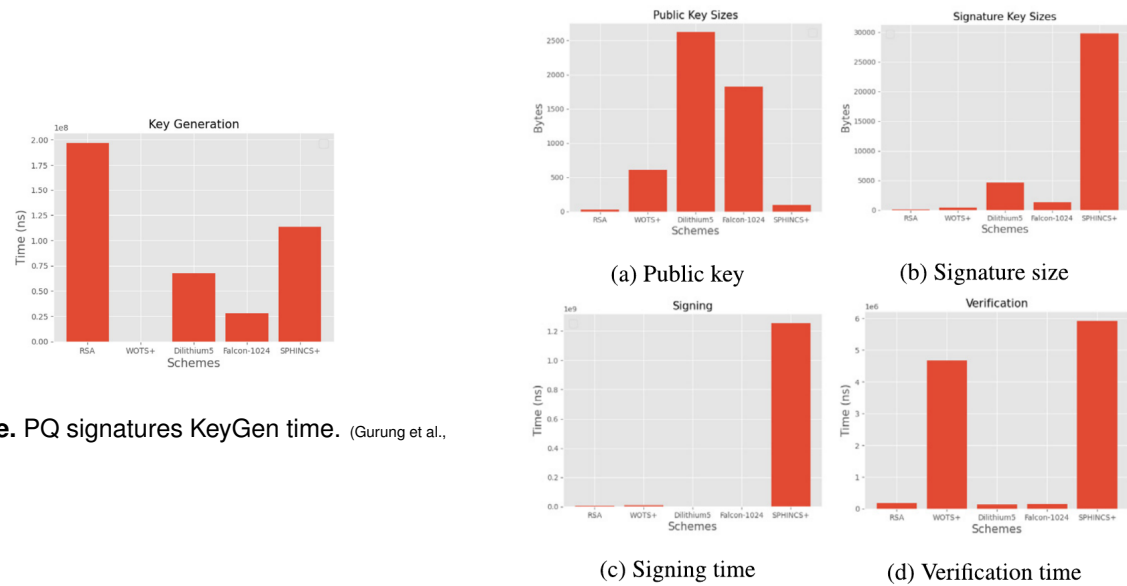


Figure. PQ signatures KeyGen time. (Gurung et al., 2024)

Figure. PQ Signatures times and sizes comparison. (Gurung et al., 2024)

FORMAL VERIFICATION

INTRODUCTION

► Symbolic Formal Verification

- Verifies protocol specifications in an abstract adversarial model
- Checks secrecy and authentication properties
- Tools: Tamarin, ProVerif, VerifPal

► Game-based Formal Verification

- Computational security via reduction-based proofs
- Security defined through adversarial games
- Tools: EasyCrypt, CryptoVerif

► Implementation Certification

- Ensures correctness of concrete implementations
- Uses proof assistants and verified toolchains
- Tools: Why3 Framework, Coq, F*, Isabelle

► UC Proofs

- Simulation-based composable security framework
- Proves real protocol emulates an ideal functionality

FORMAL VERIFICATION

TAMARIN PROVER

- ▶ Abstract protocol modeling (no computational assumptions)
- ▶ Multiset rewriting rule → Updating set of Facts through actions.
- ▶ Trace-based induction by writing Lemmas
- ▶ Dolev–Yao attacker Adversary model
 - full network control
 - Optional key compromise and state corruption
- ▶ Used to prove
 - Properties such as Secrecy, authentication, and forward secrecy properties
 - Protocols such as
 - ▶ Apple's iMessage PQ3 protocol⁵,
 - ▶ Signal Messaging Protocol⁶,
 - ▶ SwissPost Voting Framework⁷

⁵Linker and Sasse, 2024

⁶Beguinet et al., 2023

⁷Baloglu et al., 2025

FORMAL VERIFICATION

GAME-BASED PROOFS

- ▶ Computational security⁸ via reduction-based proofs⁹
- ▶ Security defined as a game between adversary and challenger
- ▶ Sequence-of-games technique to bound adversarial advantage
- ▶ Explicit hardness assumptions (e.g., EUF-CMA¹⁰, IND-CPA¹¹)
- ▶ Tools:
 - EasyCrypt: Most used, various add-ons, supports various inductions
 - CryptoVerif: More automated than EasyCrypt | Automated sequence-of-games transformations
 - Coq: General-purpose theorem prover | Requires building cryptographic framework manually
- ▶ Used to prove: SPHINCS+¹², XMSS¹³, KZG¹⁴

⁸"No efficient adversary can break this with non-negligible probability."

⁹"If an attacker can break our protocol, then we could use that attacker to break some well-known hard problem."

¹⁰Existential Unforgeability under Chosen Message Attack

¹¹Indistinguishability under Chosen Plaintext Attack

¹²Barbosa et al., 2024

¹³Barbosa et al., 2023

¹⁴Palak and Haines, 2025

FORMAL VERIFICATION

IMPLEMENTATION VERIFICATION

- ▶ Connecting proofs to implementation
- ▶ Eliminates entire classes of bugs.
- ▶ Provides machine-checkable, high-assurance guarantees.
- ▶ Frameworks
 - Why3 – Generates verification conditions and connects to multiple SMT solvers.
 - Frama-C – Deductive verification of C programs.
 - F* – Verification-oriented language for cryptographic implementations (used in HACLS*).

FORMAL VERIFICATION

UC PROOFS

- ▶ A simulation-based security framework ¹⁵ ensuring protocols remain secure even when composed with arbitrary other protocols.
- ▶ Models Real World, Ideal World, Simulator¹⁶ and Environment¹⁷
- ▶ Security is defined by comparing a real-world execution to an ideal-world execution.
- ▶ Tools for UC Proofs
 - EasyCrypt / UC Crypt expansion
 - CryptoVerif
 - Isabelle/HOL
 - Coq

¹⁵Introduced by Canetti (2001)

¹⁶Mimics the adversary's view in the ideal world.

¹⁷Attempts to distinguish real vs ideal executions.

Part II

HASHSIGS SYMBOLIC VERIFICATION

TAMARIN PROOF

PROGRESS

- ▶ Simplified version
 - Hash length = 16
 - chunks length = $4 + 2$.
- ▶ Modeled building blocks such as
 - Only Hashing builtin
 - Symbolic natural numbers
 - Bit Decomposition
 - Chunking and Base- w Conversion
 - Hash-chains
 - Checksum integration
 - KeyGen, Sign and Verify procedures.
- ▶ Security Restrictions and Invariants
 - Only one signer instance.
 - Single root-segment generation.
 - Single message per signing session.
 - Bit Decomposition correctness constraints.
 - Bounded hash-chain length.

TAMARIN PROOF

CODE DEMO

```
// Grouping every w-bits
rule rule_bits_to_wBitNumber:
  let
    sj0 = succ(j0)
    sj1 = succ(sj0)
    sj2 = succ(sj1)
    chunkIndex = div4(j0)
    chunkValue = to_w(b0, b1, b2, b3)
  in
  [
    ValuedMessageBit(m, b0, j0),
    ValuedMessageBit(m, b1, sj0),
    ValuedMessageBit(m, b2, sj1),
    ValuedMessageBit(m, b3, sj2),
    !Signer(~seed, pSeedS)
  ]
  --[
    Signer_StartedChunkAt(~seed, m, chunkIndex),
    Signer_GroupedBits(chunkIndex, b0, b1, b2, b3),
    Signer_ComputedChunkValue(chunkIndex, chunkValue)
  ]->
  [ !MessageChunk(m, chunkIndex, chunkValue)]
// restriction to control grouping to start every 4 bits
restriction rest_w divisor_first bit: "All seed m chunkIndex #i1.
Signer_StartedChunkAt(seed, m, chunkIndex)@i1 ==> not(chunkIndex = INF)"
```

Figure. Chunking rule code from Tamarin proof sketch

TAMARIN PROOF

VISUALIZATION DEMO

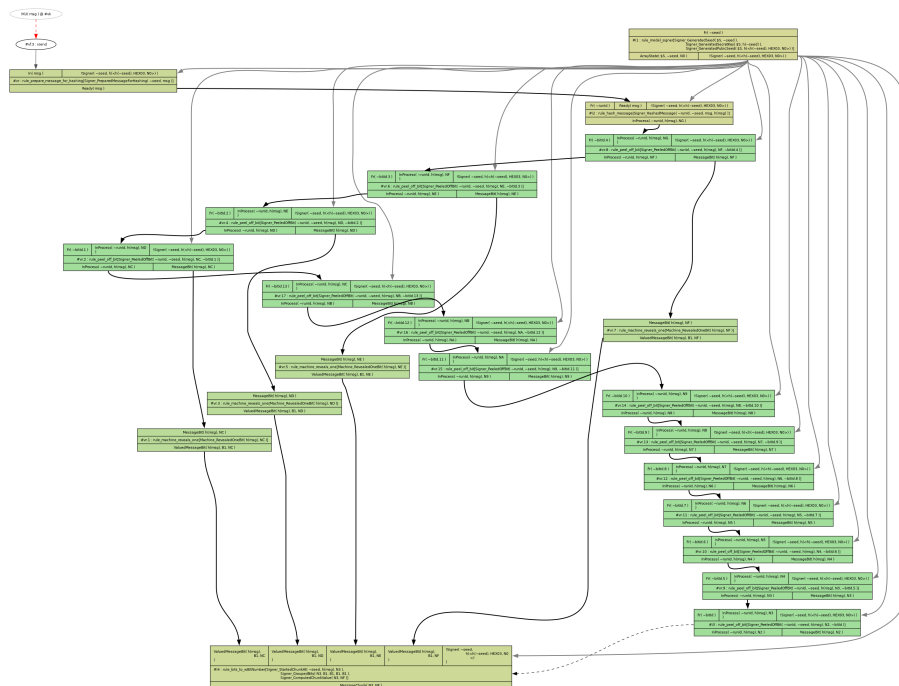


Figure. Trace visualization sample (chunking).

TAMARIN PROOF

NEXT STEPS

- ▶ Induction Oracle manipulation
- ▶ Detailed Modeling
 - Multi user setting
 - Linking Hash and Bit-Decomposition
 - Linking checksum
 - Adversary Modeling
 - ▶ Adaptive Querying ¹⁸.
 - ▶ Attack Trace Exploration
- ▶ Explicit Lemmas
 - correctness lemma: honest signatures verify.
 - soundness lemma: forged signatures are impossible.
 - Private-key Public-Key Binding Lemma
 - Explicit EUF-CMA Security Lemma¹⁹

¹⁸We need it for EUF-CMA setting

¹⁹An adversary cannot produce a valid signature on a fresh message without access to the secret key.

Part III

HASHSIGS GAME-BASED VERIFICATION

WOTS+ EASYCRYPT PROOF

MAIN POINTS (I) ²¹

- ▶ Multi-instance EUF-GCMA²⁰ security via a sequence of game-based reductions
- ▶ Proof Strategy Summary
 - Define WOTS-TW scheme and EUF-GCMA experiment.
 - Inline oracle behavior and normalize sampling.
 - Perform structured game hops.
 - Reduce adversary advantage to underlying NPRF and hash-chain distinguishability.
 - Establish overall security bound via reduction.
- ▶ Reduction Modules
 - reduces forgery to distinguishing structured vs. random hash chains.
 - structured multi-domain tweakable hash reductions.
 - Oracle variants used to bridge intermediate games.

²⁰ Existential Unforgeability under Generic Chosen-Message Attack

²¹ Barbosa et al., 2024

WOTS+ EASYCRYPT PROOF

GAMES, HYBRIDS, REDUCTIONS²²

G_0 : Real Multi-instance EUF-GCMA Experiment

↓ H0: (Oracle Inlining)

G_1 : Oracle Inlining / Normalization

↓ H1: (PRF $\rightarrow$ Random)

G_2 : PRF-to-Random Replacement (NPRF Reduction)

↓ H2: (Hash-Chain Abstraction)

G_3 : Random Chain Simulation $\rightarrow$ H3: (Bad Event Isolation) $\rightarrow G'_3$: Collision Handling Variant

↓ H4: (Final Idealization)

G_4 : Final Reduction Game

Bounding Step: Overall advantage bounded by PRF advantage (NPRF assumption) and Hash-chain distinguishability advantage.

²²Barbosa et al., 2024

TOWARD HASHSIGS PROOF

INITIAL STEPS

- ▶ WOTS+ EasyCrypt proof documentation
- ▶ Restructure the proof
 - WOTS+ proof security under independent secrets.
 - Prove PRF expansion is indistinguishable from independent sampling.
 - Composition of WOST+ and HashSigs modifications into final theorem.

TOWARD HASHSIGS PROOF

PRIVATEKEY

► WOTS+

$$sk = (x_1, \dots, x_\ell), \quad x_i \stackrel{i.i.d.}{\leftarrow} \{0, 1\}^n.$$

- Each secret element is sampled independently and uniformly.
- Independence is built directly into the probabilistic model.

► HashSigs

$$\text{seed} \leftarrow \{0, 1\}^n, \quad x_i = \text{PRF}(\text{seed}, i).$$

- Only a single seed is stored.
- Secret elements are deterministically derived.
- Independence relies on pseudorandomness of the PRF.

TOWARD HASHSIGS PROOF

REQUIRED PRIVATEKEY MODIFICATIONS

► Signing Oracle Adjustment

- Signing no longer reads stored x_i values.
- Instead, it recomputes:

$$x_i = \text{PRF}(\text{seed}, i)$$

- The reduction must simulate PRF evaluations consistently across queries.

► State Representation Update

- Secret key becomes a single seed instead of a list.
- Signing oracle must be deterministic with respect to PRF outputs.
- Proof state must track PRF oracle consistency.

► Tightness Adjustment

- Original bound:

$$\text{Adv}_{\text{EUF}} \leq \ell \cdot \text{Adv}_{\text{OWF}}$$

- New bound:

$$\text{Adv}_{\text{EUF}} \leq \text{Adv}_{\text{PRF}} + \ell \cdot \text{Adv}_{\text{OWF}}$$

- Slight loss in tightness due to hybrid transition.

TOWARD HASHSIGS PROOF

PUBLICKEY

► WOTS+

$$pk = (pk_1, \dots, pk_\ell), \quad pk_i = F^{w-1}(x_i).$$

- Public key is an explicit vector of chain endpoints.
- Verification checks equality component-wise.
- No compression step.

► HashSigs

$$pk = H(pk_1 \parallel \dots \parallel pk_\ell).$$

- Chain endpoints are concatenated and hashed.
- Public key is a single digest.
- Verification recomputes endpoints, hashes them, and compares the resulting digest.

TOWARD HASHSIGS PROOF

REQUIRED PUBLICKEY MODIFICATIONS

- ▶ Redefine the public key in the security experiment as:

$$pk = H(pk_1 \parallel \dots \parallel pk_\ell).$$

- ▶ Proof verification flow modification
 - Recompute all endpoints.
 - Hash the concatenation.
 - Compare to the stored compressed public key.
- ▶ Collision-resistance assumption for H on fixed-length inputs.
- ▶ New hybrid arguments
 - Hybrid A: Compressed public key.
 - Hybrid B: Explicit vector public key.
- ▶ New security bound

$$\text{Adv}_{\text{EUF}} \leq \text{Adv}_{\text{PRF}} + \ell \cdot \text{Adv}_{\text{OWF}} + \text{Adv}_{\text{CR}}.$$

TOWARD HASHSIGS PROOF

RANDOMIZATION ELEMENTS & MASKS

► WOTS+

- Assumes each (i, j) pair receives an independent pseudorandom mask.
- Masks may be conceptually pre-sampled or lazily replaced via hybrids.
- Requires an explicit PRF security assumption.
- Randomization elements may not be stored explicitly; they are derived abstractly.
- Assumes perfect independence and correct domain separation.

► HashSigs

- Independence is achieved only if ADRS fields are correctly updated per chain and step.
- Masks are computed on-the-fly during each chain step
- Relies concretely on the chosen hash/PRF primitive behaving pseudorandomly.
- Only the seed is stored; masks are derived deterministically from it.
- Security depends on correct ADRS updates, no index reuse, and correct PRF invocation.

Part IV

SUMMARY

TAMARIN SYMBOLIC PROOF

MILESTONES

Table. HashSigs **Tamarin proof** Milestones

version	Features	Time estimation
v0.4	Initial Sketch End-to-End Executability lemma	Ready
v0.5	Soundness & Completeness Lemma Induction Oracle Sketch	Feb-W4
v0.6	Bit Decomposition Binding	Feb-W4
v0.7	Checksum Binding	Mar-W1
v0.8	Multi-user setup	Mar-W2
v0.9	Adversary Modeling	Mar-W3
v1.0	Protocol improvements	Mar-W4

PROOF PAPER

MILESTONES

Table. HashSigs **Proof Paper** Milestones

version	Features	Time estimation
v0.4	Literature Review Sketch Notations & Protocol Sketch	Ready
v0.5	Complete Protocol and notations	Feb-W4
v0.6	Tamarin Proof section sketch	Mar-W1
v0.7	Threat Modeling section Detailed Tamarin Modeling	Mar-W1
v0.8	Properties & Lemmas section sketch	Mar-W2
v0.9	Cleanups Complete lemmas	Apr-W1
v1.0	Peer review Addressing comments	Apr-W3

EASYCRYPT GAME-BASED PROOF

MILESTONES ²³

Table. HashSigs **EasyCrypt proof** Milestones

version	Features	Time estimation
v0.1	Initial analysis of WOTS+ easycrypt code	Ready
v0.2	Writing a documentation over their proof	Mar-W1
v0.3	Proof Structure Refactoring	Mar-W2
v0.4	PrivateKey Modifications	Mar-W3
v0.5	Randomization Elements Modifications	Mar-W3
v0.6	PublicKey Modifications	Mar-W3
v0.7	Sign flow Modifications	Apr-W1
v0.8	Verify flow Modifications	Apr-W1
v0.9	Testing and visualizing the whole proof	Apr-W3
v1.0	Final report	Apr-W4

²³In case we want to drop Tamarin Proof and switch to EasyCrypt Proof

REFERENCES I

-  Baloglu, S., Bursuc, S., Gil-Pons, R., & Mauw, S. (2025). **Towards formal verification and corrupted setup security for the SwissPost voting system.**
-  Barbosa, M., Dupressoir, F., Grégoire, B., Hülsing, A., Meijers, M., & Strub, P.-Y. (2023). **Machine-checked security for XMSS as in RFC 8391 and SPHINCS⁺.**
https://doi.org/10.1007/978-3-031-38554-4_14
-  Barbosa, M., Dupressoir, F., Hülsing, A., Meijers, M., & Strub, P.-Y. (2024). **A Tight Security Proof for SPHINCS+, Formally Verified.** Retrieved January 15, 2026, from <https://eprint.iacr.org/2024/910>
Publication info: Preprint.
-  Beguinet, H., Chevalier, C., Ricosset, T., & Senet, H. (2023, October). **Formal Verification of a Post-Quantum Signal Protocol with Tamarin.** In B. B. Hedia, Y. Maleh, & M. Krichen (Eds.), *LNCS* (pp. 105–121, Vol. 14368). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-49737-7_8
-  Gan, L., & Yokubov, B. (2023). **A Performance Comparison of Post-Quantum Algorithms in Blockchain.** *The Journal of The British Blockchain Association*, 6(1), 1–10.
[https://doi.org/10.31585/jbba-6-1-\(1\)2023](https://doi.org/10.31585/jbba-6-1-(1)2023)
-  Gurung, D., Pokhrel, S. R., & Li, G. (2024). **Performance analysis and evaluation of postquantum secure blockchained federated learning.** *Computer Networks*, 255, 110849.
<https://doi.org/https://doi.org/10.1016/j.comnet.2024.110849>
-  Linker, F., & Sasse, R. (2024). **A Formal Analysis of Apple's iMessage PQ3 Protocol.**
<https://eprint.iacr.org/2024/1395>

REFERENCES II

-  Palak & Haines, T. (2025). **Formalisation of the KZG polynomial commitment schemes in EasyCrypt.**
-  Srivastava, V., Baksi, A., & Debnath, S. K. (2023). ***An Overview of Hash Based Signatures***. Retrieved January 15, 2026, from <https://eprint.iacr.org/2023/411>
Publication info: Preprint.